

CodePoster

Rapport technique d'ingénierie

Jeu de Programmation Multijoueur en Temps Réel

Analyse technique architectural

28 mai 2026

Version 1.0 – MVP

Statut	Version courante du projet
Équipe	Aymane EDDAMANE & Taha HOUMMADI
Diffusion	Rapport enseignant

Table des matières

1	Résumé exécutif	4
2	Contexte et objectifs fonctionnels	4
2.1	Objectif produit	4
2.2	Principaux cas d’usage	4
2.3	Déroulement d’une partie	5
2.4	Conditions de victoire	5
2.5	Les 5 tâches de débogage Python	5
3	Architecture globale	5
3.1	Vue d’ensemble	6
3.2	Principes clés	6
4	Backend – Analyse technique	6
4.1	Technologies	6
4.2	Structure des packages	6
4.3	Flux principal	7
4.4	Sandbox d’exécution	7
4.5	Bugs identifiés dans le code source	7
5	Frontend – Analyse technique	8
5.1	Technologies	8
5.2	Navigation	8
5.3	Collaboration temps réel	8
5.4	UI et expérience utilisateur	8
6	Modèle de données	9
6.1	Entités principales	9
6.2	Stratégie de persistance	9
7	Flux temps réel (STOMP)	9
7.1	Destinations client → serveur (/app)	9
7.2	Destinations serveur → client (/topic, /queue)	9
7.3	Diagramme de séquence – Cycle d’une partie	10
8	Sécurité et risques	10
8.1	État actuel	10
8.2	Risques	11
8.3	Recommandations	11
9	Qualité logicielle et tests	11
9.1	État actuel	11
9.2	Recommandations	11
10	Performance et scalabilité	11
11	Observabilité et monitoring	12
11.1	État actuel	12
11.2	Plan d’amélioration	12

12 Nouvelles fonctionnalités proposées	12
12.1 Service de collecte de données (Analytics)	12
12.1.1 Valeur de marché	12
12.1.2 Données collectables	12
12.1.3 Architecture recommandée	13
12.1.4 Implémentation recommandée	13
12.2 Assistant IA intégré	13
12.2.1 Modes d'utilisation	14
12.2.2 Architecture du hint engine	14
12.2.3 Stack recommandée	14
12.2.4 Diagramme d'intégration des deux nouveaux services	14
13 Améliorations et feuille de route	14
13.1 Court terme – Corrections prioritaires	15
13.2 Moyen terme – Nouvelles fonctionnalités	15
13.3 Long terme – Évolutions majeures	15
14 Annexes	15
14.1 Technologies (résumé)	15
14.2 Glossaire	16

1 Résumé exécutif

CodePoster est un jeu de programmation multijoueur en temps réel. Jusqu'à 4 joueurs se connectent à une salle, reçoivent un rôle secret (*crewmate* ou *imposter*) et réparent en parallèle des fichiers Python corrompus sous contrainte de temps. Le système s'appuie sur :

- un backend Java Spring Boot (WebSocket/STOMP + REST) ;
- un frontend React/Vite avec UI riche et animations ;
- une collaboration P2P via Yjs + WebRTC ;
- MongoDB pour la persistance des joueurs ;
- Redis pour le cache des listes de joueurs par salle ;
- un sandbox Docker pour exécuter du code Python.

- Collaboration P2P native via Yjs CRDT + WebRTC – aucune charge serveur pour la synchronisation du code.
- Sandbox Docker isolé avec limites CPU/mémoire – bonne maîtrise des risques d'exécution.
- Stack événementielle cohérente : STOMP sur WebSocket pour tous les événements de jeu.
- Technologies récentes : Java 21, Spring Boot 4.0.3, React 19, Vite 7, Tailwind 4.

- Credentials MongoDB codés en dur dans `application.properties`.
- Test runner Python codé en dur (`pattern_match`) – pas encore généralisé aux 5 tâches.
- `readyNumber()` globale – bug potentiel en multi-salle.
- Pas d'authentification ni de gestion de session persistante.
- Serveur de signaling Yjs public (`yjs.dev`) – dépendance externe non contrôlée.

2 Contexte et objectifs fonctionnels

2.1 Objectif produit

Proposer une expérience ludique et coopérative de debugging/programmation, avec un rôle d'imposteur qui sabote la progression de l'équipe. L'ambiance visuelle cyber/futuriste renforce l'immersion avec des animations shader, des effets glitch et des transitions Lottie.

2.2 Principaux cas d'usage

- Créer ou rejoindre une salle.
- Discussion en temps réel (chat).
- Réparation de 5 fichiers de code en parallèle.
- Sabotage, réunions d'urgence, votes d'exclusion.

- Vote de compilation et résultat de build.
- Révélation du rôle et fin de partie.

2.3 Déroulement d'une partie

1. Création ou rejoindre une salle via le Lobby.
2. Attribution aléatoire d'un rôle secret : crewmate (3) ou imposteur (1).
3. Lancement simultané de 5 tâches de débogage Python.
4. Les crewmates réparent les fichiers en collaboration (éditeur Yjs partagé).
5. L'imposteur sabote la progression d'un joueur.
6. N'importe quel joueur peut déclencher une réunion d'urgence.
7. Vote d'exclusion à la majorité – le joueur le plus voté est éjecté.
8. Révélation du rôle et annonce du résultat.

2.4 Conditions de victoire

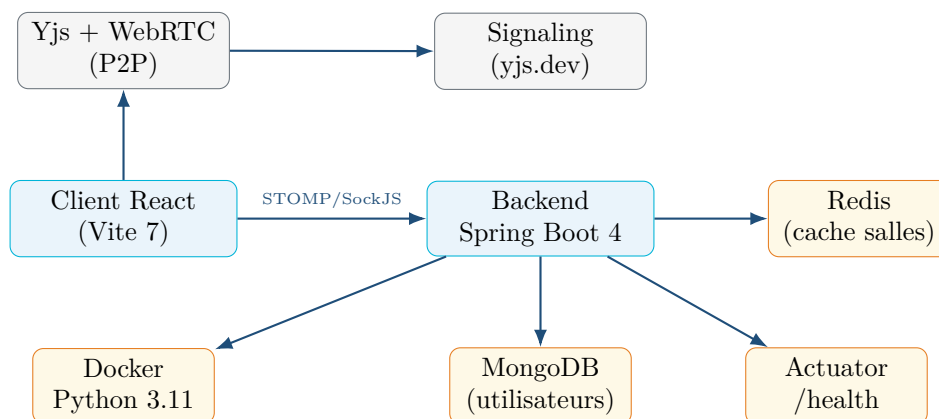
Condition	Vainqueur	Déclencheur
Tous les crewmates finissent leurs tâches	Crewmates	<code>handleTaskComplete()</code>
L'imposteur est éjecté par vote	Crewmates	<code>ejectedIsImposter = true</code>
Vote éjecte un crewmate	Imposteur	<code>ejectedIsImposter = false</code>
Timer expire	Imposteur	<code>handleGameOver()</code>

2.5 Les 5 tâches de débogage Python

Clé fichier	Description attendue
<code>neural_hash</code>	Implémentation d'une fonction de hachage
<code>data_sort</code>	Algorithme de tri de données
<code>auth_check</code>	Vérification d'authentification / tokens
<code>key_rotation</code>	Rotation de clés cryptographiques
<code>grid_scan</code>	Parcours / analyse de grille 2D

3 Architecture globale

3.1 Vue d'ensemble



3.2 Principes clés

- **Temps réel** : STOMP au-dessus de WebSocket pour les événements de jeu.
- **Collaboration P2P** : Yjs CRDT pour la synchronisation de texte sans charge serveur.
- **Persistant + cache** : MongoDB (utilisateurs) + Redis (listes par salle).
- **Sandbox d'exécution** : Docker exécute le code Python en isolation totale.

4 Backend – Analyse technique

4.1 Technologies

Technologie	Version	Rôle
Java	21 (LTS)	Langage principal
Spring Boot	4.0.3	Framework applicatif
Spring WebSocket + STOMP	–	Messagerie temps réel
Spring Data MongoDB	–	Persistance utilisateurs
Spring Data Redis	–	Cache listes de joueurs
Spring Boot Actuator	–	Health check /actuator/health
Lombok	Stable	Réduction boilerplate Java
Docker (python :3.11-slim)	Runtime	Sandbox d'exécution isolé

4.2 Structure des packages

```

com.example.websocketchat
|-- config/
| |-- WebSocketConfig.java (endpoint /ws, prefixes /app /topic /queue)
| |-- RedisConfig.java
| |-- CustomHandshakeInterceptor (injection sessionId)
| '-- WebSocketEventListener (gestion deconnexion)
|-- controller/
| |-- RoomController.java (join, vote, sabotage, task, game-over)
| |-- ChatController.java (messages salle d'attente)
| |-- CompileController.java (REST POST /api/compile)
| '-- UserController.java
|-- service/
| |-- RoomService.java (roles, votes, taches, fin de partie)

```

```
| |-- CompilerService.java (execution Python via Docker)
| |-- MessagingService.java (sendToTopic / sendToUser)
| '-- userService.java (CRUD + cache Redis)
|-- model/
| |-- User.java (@Document MongoDB)
| |-- ChatMessage.java (types: CHAT, VOTE, SABOTAGE, ...)
| |-- Task.java / TestCase.java
| |-- Submission.java
| '-- Sabotage.java
'-- repository/
    |-- UserRepository.java
    '-- TaskRepository.java
```

4.3 Flux principal

1. Connexion STOMP avec identifiant utilisateur.
2. Ajout du joueur en salle (attente ou jeu).
3. Attribution des rôles et début de partie.
4. Gestion des événements (chat, sabotage, vote).
5. Fin de partie (vote ou timer).

4.4 Sandbox d'exécution

Le service de compilation implémente un sandbox sécurisé :

```
// CompilerService.java -- Parametres Docker
"docker", "run",
    "--rm", // suppression auto
    "--network", "none", // reseau desactive
    "--memory", "100m", // limite RAM
    "--cpus", "0.5", // limite CPU
    "-v", tempDir + ":/app",
    "python:3.11-slim",
    "python", "test_runner.py"
// Timeout : 3 secondes (process.waitFor)
```

Le `test_runner.py` est codé en dur pour tester `pattern_match()` uniquement. Il doit être généralisé pour charger dynamiquement les cas de test des 5 tâches depuis `TaskRepository`.

4.5 Bugs identifiés dans le code source

La méthode `userService.readyNumber()` compte **tous** les utilisateurs *ready* en base, sans filtre par salle. En contexte multi-salle, cela pourrait déclencher prématurément une partie.

Correction : ajouter un filtre `roomId` dans la requête MongoDB.

La méthode `findByRoom()` est annotée `@CachePut`, ce qui signifie qu'elle écrit toujours en cache mais ne lit pas depuis le cache. Changer en `@Cacheable` pour éviter des requêtes Mongo inutiles à chaque appel.

5 Frontend – Analyse technique

5.1 Technologies

Technologie	Version	Rôle
React	19.2.0	Librairie UI
React Router	7.13.0	Navigation SPA
Vite	7.2.4	Build tool
Tailwind CSS	4.1.18	Styling utilitaire
GSAP + @gsap/react	3.14.2	Animations avancées
Three.js + @react-three/fiber	0.182.0 / 9.5.0	Rendus 3D / shaders
Lottie-react	2.4.1	Animations JSON
SockJS-client + stompjs	1.6.1 / 2.3.3	Client WebSocket STOMP
Yjs	13.6.30	CRDT collaboration temps réel
y-webrtc	10.3.0	Transport WebRTC pour Yjs
lucide-react	0.563.0	Icônes

5.2 Navigation

- `/login` – LoginPage (connexion, choix username).
- `/` – LobbyPage (créer / rejoindre salle).
- `/create` – CreateRoomPage.
- `/waiting/:roomId` – WaitingRoomPage (prêt, affichage joueurs).
- `/room/:roomId` – RoomPage (jeu complet).
- `/role` – RolePage (révélation du rôle secret).
- `/reveal` – RevealPage (fin de partie).
- `/profile` – ProfilePage (stats – statique pour l'instant).

5.3 Collaboration temps réel

- Yjs crée un document partagé (`Y.Doc`) par salle.
- Chaque fichier est un `Y.Text` synchronisé P2P automatiquement.
- Awareness partage l'activité (fichier actif, typing).
- Mode *Live* permet de regarder un joueur en lecture seule.

5.4 UI et expérience utilisateur

- Design cyber / futuriste cohérent sur l'ensemble des pages.
- Effets glitch, shader Three.js et neural network animé sur la page d'accueil.

- Animations Lottie contextuelles : emergency, compile, countdown, recording.
- Panels modulaires : activité en direct, sabotage, vote, compilation.
- Sons d’ambiance (`lightning.mp3`) pour les événements critiques.

6 Modèle de données

6.1 Entités principales

- **User** (@Document MongoDB) : `username`, `waitingroomId`, `ready`, `Imposter`, `roomOwner`, `sessionId`.
- **ChatMessage** : `type` (CHAT, JOIN, LEAVE, READY, START_GAME, TASK_COMPLETE, VOTE, VOTE_RESULT, SABOTAGE, GAME_OVER, EMERGENCY_MEETING, COMPILE_VOTE_START...), `content`, `sender`, `target`, `sabotage`.
- **Task** / **TestCase** : modèles existants, non encore chargés dynamiquement.
- **Submission** : fichiers envoyés pour compilation.

6.2 Stratégie de persistance

Données	Stockage	Durée de vie
Utilisateurs	MongoDB Atlas	Session (pas de TTL défini)
Listes de joueurs	Redis Cache	Jusqu’à invalidation
Votes en cours	ConcurrentHashMap	Durée du vote
Progression tâches	ConcurrentHashMap	Durée de la partie
Code collaboratif	Yjs P2P (mémoire)	Session WebRTC
Tâches de débogage	Hardcodé frontend	N/A (à migrer)

7 Flux temps réel (STOMP)

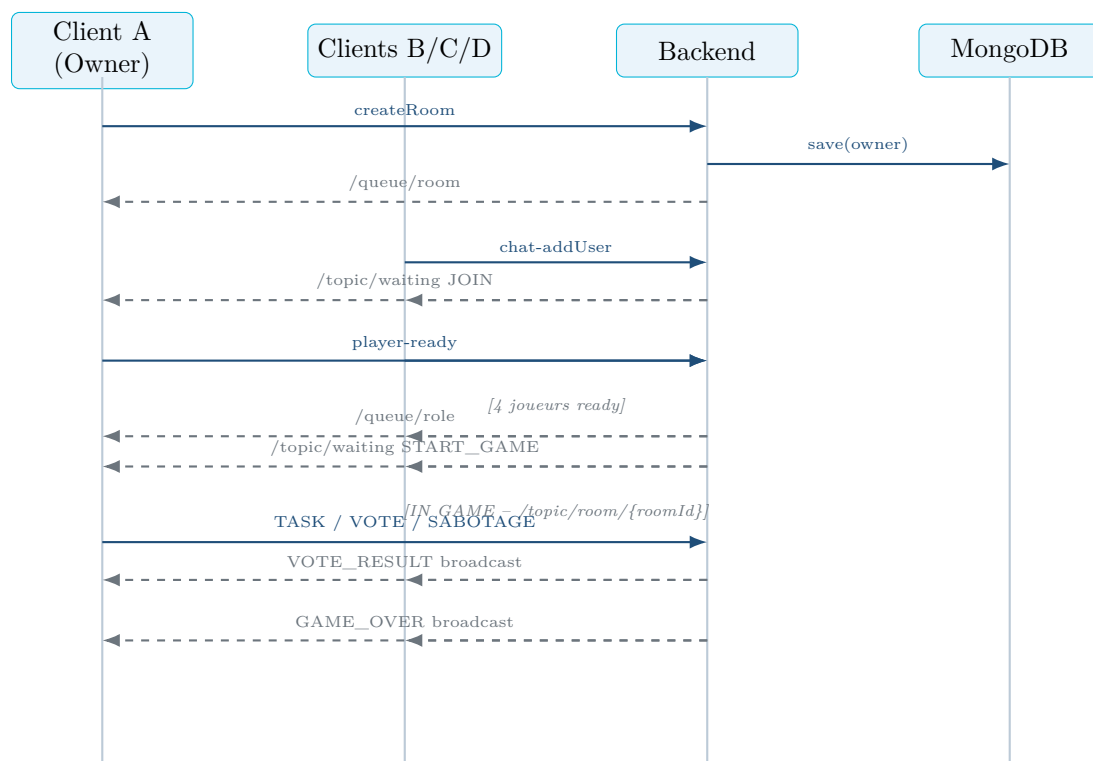
7.1 Destinations client → serveur (/app)

Destination	Rôle
<code>/app/createRoom/{roomId}</code>	Création de salle
<code>/app/chat-addUser/{roomId}</code>	Rejoindre salle d’attente
<code>/app/player-ready/{roomId}</code>	Indiquer prêt
<code>/app/room/{roomId}/join</code>	Rejoindre salle de jeu
<code>/app/room/{roomId}/chat</code>	Chat en partie
<code>/app/room/{roomId}/task</code>	Tâche terminée
<code>/app/room/{roomId}/vote</code>	Vote d’éjection
<code>/app/room/{roomId}/sabotage</code>	Sabotage
<code>/app/room/{roomId}/emergency</code>	Réunion d’urgence
<code>/app/room/{roomId}/compile-vote-start</code>	Initier un vote de compilation
<code>/app/room/{roomId}/compile-vote</code>	Voter pour la compilation
<code>/app/room/{roomId}/game-over</code>	Fin de partie (timer)

7.2 Destinations serveur → client (/topic, /queue)

Destination	Type	Description
/topic/waiting/{roomId}	Public	Salle d'attente (JOIN, READY, START_GAME)
/topic/room/{roomId}	Public	Tous les événements de jeu
/queue/role	Privé	Rôle secret assigné
/queue/room	Privé	Confirmation création salle
/queue/errors	Privé	Erreurs (salle pleine, username pris)
/queue/room/{roomId}/sabotage	Privé	Notification de sabotage ciblé

7.3 Diagramme de séquence – Cycle d'une partie



8 Sécurité et risques

8.1 État actuel

Point	État	Risque
Credentials MongoDB	URI en clair dans <code>application.properties</code>	CRITIQUE
Authentification	Aucune (username libre)	ÉLEVÉ
Autorisation STOMP	Aucune	ÉLEVÉ
CORS	Limité à <code>localhost:5173</code>	Faible (dev)
Exécution Python	Docker isolé, <code>-network none</code>	Maîtrisé
Signaling Yjs	Serveur public <code>yjs.dev</code>	MOYEN
Fichiers temp	Non supprimés après exécution	MOYEN

8.2 Risques

- Accès non autorisé aux salles.
- Exposition de secrets (DB).
- Scalabilité limitée du broker STOMP simple.

8.3 Recommandations

Déplacer l'URI MongoDB dans des variables d'environnement :

```
spring.mongodb.uri=${MONGODB_URI}
```

Ne jamais commiter des credentials en clair dans un dépôt Git.

- Implémenter un système JWT (stateless, adapté au WebSocket).
- Valider côté backend que l'expéditeur STOMP correspond au `sessionId`.
- Limiter la création de salles aux utilisateurs authentifiés.

9 Qualité logicielle et tests

9.1 État actuel

Catégorie	Couverture	Commentaire
Tests backend	Minimale	Uniquement chargement de contexte Spring
Tests de services	Absents	RoomService, CompilerService non testés
Tests STOMP	Absents	Aucun test de flux temps réel
Tests frontend	Absents	Aucun test unitaire ou E2E
Linting	ESLint	<code>eslint.config.js</code> présent
CI/CD	Absent	Aucun pipeline d'intégration continue

9.2 Recommandations

- **Tests unitaires** (JUnit 5 + Mockito) : `RoomService.handleVote()`, `handleTaskComplete()`, `assignRoles()`, `CompilerService`.
- **Tests d'intégration WebSocket** : connexion STOMP, vote complet, fin de partie par timer.
- **Tests E2E frontend** (Playwright) : flux complet login → jeu → révélation.

10 Performance et scalabilité

- STOMP simple broker = adapté à un petit nombre de salles.
- Yjs en P2P limite la charge serveur mais dépend du réseau.
- Compilation Docker impacte CPU/RAM (limites déjà en place).

Pistes d'amélioration :

- Remplacer le SimpleBroker STOMP par Redis Pub/Sub ou RabbitMQ pour le multi-instance.
- Mettre en file les compilations (BlockingQueue ou RabbitMQ).
- Configurer un TTL Redis pour nettoyer les salles terminées.
- Ajouter des index MongoDB sur `waitingroomId` et `sessionId`.

11 Observabilité et monitoring

11.1 État actuel

- Actuator expose un endpoint de santé (`/actuator/health`) avec détails MongoDB.
- Logs `System.out.println` pour connexions WebSocket et erreurs (non structurés).
- Aucun dashboard ni alerting configuré.

11.2 Plan d'amélioration

Outil	Rôle	Priorité
SLF4J + Logback	Logs structurés avec niveaux	Haute
Prometheus + Micrometer	Métriques applicatives	Moyenne
Grafana	Dashboards temps réel	Moyenne
OpenTelemetry	Tracing distribué	Basse
Sentry (frontend)	Capture d'erreurs JavaScript	Moyenne

12 Nouvelles fonctionnalités proposées

Cette section présente deux extensions majeures envisagées pour les prochaines versions de **CodePoster** : un service de collecte et d'analyse de données, et un assistant IA intégré au jeu.

12.1 Service de collecte de données (Analytics)

12.1.1 Valeur de marché

La richesse de **CodePoster** réside dans les données comportementales qu'il génère : des indicateurs de **codage sous pression sociale**, rares et précieux pour plusieurs marchés cibles.

Marchés potentiels

- **EdTech** : identifier quels concepts Python posent le plus de difficulté aux étudiants.
- **RH / Recrutement** : évaluation comportementale sous contrainte de temps et de stress social.
- **Gaming analytics** : modélisation et segmentation du comportement joueur.
- **Recherche académique** : patterns de collaboration en codage distribué.

12.1.2 Données collectables

Par partie :

- Durée totale, résultat (`crewmates` / `imposteur`), difficulté sélectionnée.

- Nombre de votes, sabotages, réunions d’urgence déclenchées.
- Fichiers compilés et résultat (ACCEPTED / WRONG_ANSWER / RUNTIME_ERROR / TIME_LIMIT_EXCEEDED).

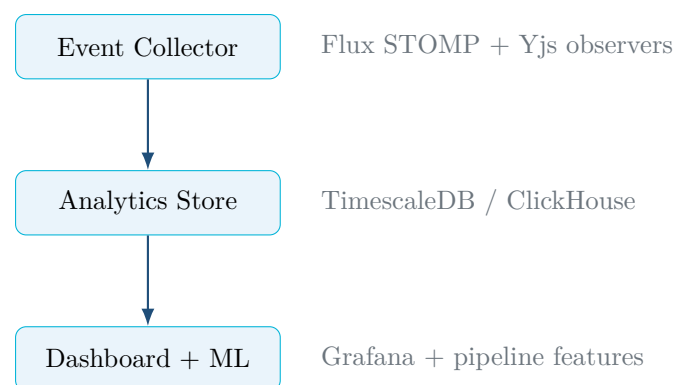
Par joueur :

- Temps passé sur chaque fichier Python (neural_hash, data_sort, etc.).
- Vitesse de frappe (keystrokes/min via Yjs Awareness).
- Nombre de modifications, corrections et erreurs de compilation.
- Pattern de navigation entre les fichiers (attention, focus).
- Taux de complétion des tâches et win rate par rôle.

Métriques sociales et comportementales :

- Pattern de vote (qui vote pour qui, fréquence de vote correct).
- Efficacité du sabotage en tant qu’imposteur.
- Comportement collaboratif mesuré via Yjs Awareness partagée.
- Fréquence des réunions d’urgence déclenchées par joueur.

12.1.3 Architecture recommandée



L’Event Collector s’intègre côté backend en écoutant tous les messages STOMP publiés sur `/topic/room/{roomId}` et les événements Yjs (via Awareness observer côté frontend) sans modifier la logique de jeu existante.

12.1.4 Implémentation recommandée

1. Ajouter un `ApplicationListener<SessionSubscribeEvent>` dans Spring pour intercepter les événements STOMP sans modifier les controllers.
2. Sérialiser chaque événement en JSON horodaté et l’insérer en base de données analytics (TimescaleDB recommandé pour les séries temporelles).
3. Côté frontend, ajouter un observer Yjs sur l’Awareness pour capturer le temps passé par fichier et la vitesse de frappe.
4. Exposer une API REST `/api/analytics/{roomId}` pour les tableaux de bord post-partie.

12.2 Assistant IA intégré

12.2.1 Modes d'utilisation

Mode	Déclencheur	Description
Hint engine	Bouton joueur	Indice contextuel sans donner la réponse
Analyse post-partie	Fin de partie	Rapport IA personnalisé par joueur
Bot imposteur	Mode solo (optionnel)	Jouer contre une IA

12.2.2 Architecture du hint engine

L'assistant IA reçoit en contexte :

- Le code actuel du joueur sur le fichier en cours.
- La description de la tâche et les cas de test attendus.
- Le rôle du joueur (crewmate) et le temps restant.

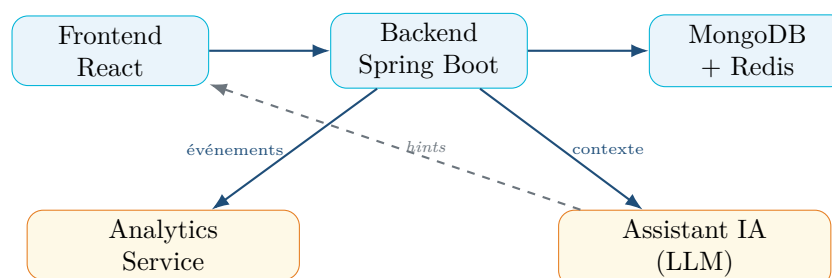
Le prompt système contraint le modèle à ne jamais donner la solution directement, mais à guider le joueur par des questions socratiques.

```
// Exemple de payload vers l'API LLM
POST /api/hint
{
  "taskKey": "auth_check",
  "currentCode": "def auth_check(token):\n ...",
  "errorOutput": "AssertionError: line 5",
  "timeRemaining": 120,
  "hintsUsed": 1
}
```

12.2.3 Stack recommandée

- **LLM Gateway** : Claude API (Anthropic) ou OpenAI GPT-4o comme modèle principal.
- **Endpoint dédié** : POST /api/hint côté Spring Boot, appelé depuis le frontend.
- **Rate limiting** : limiter à 3 indices par joueur par partie pour préserver le gameplay.
- **Analyse post-partie** : rapport structuré (JSON) converti en UI côté frontend, incluant les erreurs récurrentes, la comparaison aux autres joueurs et des conseils d'amélioration.

12.2.4 Diagramme d'intégration des deux nouveaux services



13 Améliorations et feuille de route

13.1 Court terme – Corrections prioritaires

- Sécuriser les credentials MongoDB via variables d’environnement.
- Assurer que le service de Collecte et Analyse de données fonctionnel
- Améliorer le User experience du jeu.
- Corriger `readyNumber()` pour filtrer par salle.
- Nettoyer les fichiers temporaires Docker après compilation (bloc `finally`).
- Changer `@CachePut` en `@Cacheable` sur `findByRoom()`.
- Brancher le CompilerService aux 5 tâches réelles du jeu.

13.2 Moyen terme – Nouvelles fonctionnalités

- Persister les tâches en base (`TaskRepository`).
- Implémenter l’authentification JWT.
- Ajouter les tests unitaires backend (JUnit 5).
- Configurer un pipeline CI (GitHub Actions).
- Intégrer le service de collecte de données (Event Collector).
- Persister les statistiques joueurs (profil, historique).

13.3 Long terme – Évolutions majeures

- Migration vers un broker externe (RabbitMQ / Redis Pub/Sub).
- File d’attente pour les compilations Docker.
- Serveur de signaling Yjs auto-hébergé.
- Tests E2E complets (Playwright).
- Déploiement containerisé complet (Docker Compose / Kubernetes).
- Intégration LLM (hint engine + analyse post-partie).
- Dashboard analytics Grafana + pipeline ML.

14 Annexes

14.1 Technologies (résumé)

Domaine	Technologies
Backend	Java 21, Spring Boot 4.0.3, Spring WebSocket/STOMP, Spring Data MongoDB, Spring Data Redis, Lombok, Jackson, Spring Actuator
Frontend	React 19, React Router 7, Vite 7, Tailwind CSS 4, GSAP 3, Three.js, @react-three/fiber, Lottie-react, lucide-react
Temps réel	SockJS, stompjs, Yjs (CRDT), y-webrtc (WebRTC)
Sandbox	Docker, python :3.11-slim
Persistence	MongoDB Atlas (users), Redis (cache salles)
Build/Dev	Maven (backend), Vite + npm (frontend), ESLint

14.2 Glossaire

- **STOMP** : Simple Text Oriented Messaging Protocol – protocole de messagerie au-dessus de WebSocket.
- **CRDT** : Conflict-free Replicated Data Type – structure de données pour convergence sans conflit.
- **Yjs** : Implémentation JavaScript de CRDT pour la collaboration temps réel.
- **y-webrtc** : Provider Yjs utilisant WebRTC pour la communication P2P.
- **SockJS** : Librairie JavaScript offrant un fallback WebSocket sur les navigateurs anciens.
- **Sandbox** : Environnement d'exécution isolé (Docker) pour limiter les risques.
- **SimpleBroker** : Broker STOMP en mémoire fourni par Spring – adapté au développement uniquement.
- **Crewmate** : Rôle coopératif – objectif : réparer les 5 fichiers ou identifier l'impateur.
- **Imposteur** : Rôle adversarial – objectif : saboter et ne pas être éjecté.
- **Awareness** : Fonctionnalité Yjs partageant l'état des utilisateurs connectés (fichier actif, présence).
- **TimescaleDB** : Base de données time-series open-source, extension de PostgreSQL.
- **ClickHouse** : Base OLAP haute performance pour l'analyse de gros volumes d'événements.
- **LLM** : Large Language Model – modèle de langage utilisé pour l'assistant IA.
- **TTL** : Time To Live – durée de vie d'une entrée en cache ou en base.